

Competitive Programming

Section 51 Editorial: Invariants In Hard Problems

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	3
1.1	Problem-specific proof map	3
1.2	Detailed solution guide	3
2	A. Same Parity	5
2.1	Restatement	5
2.2	Observations and algorithm	5
2.3	Correctness proof	5
2.4	Complexity	5
2.5	Full C++ solution	5
2.6	Full Python solution	5
3	B. XOR Balanced Splits	6
3.1	Restatement	6
3.2	Observations and algorithm	6
3.3	Correctness proof	6
3.4	Complexity	6
3.5	Full C++ solution	6
3.6	Full Python solution	6
4	C. Same Residue	8
4.1	Restatement	8
4.2	Observations and algorithm	8
4.3	Correctness proof	8
4.4	Complexity	8
4.5	Full C++ solution	8
4.6	Full Python solution	8
5	D. Adjacent Flip Distance	9
5.1	Restatement and first invariant	9
5.2	Forced recurrence and trace	9
5.3	Algorithm	9
5.4	Correctness proof	10
5.5	Complexity, implementation notes, and tests	10
5.6	Full C++ solution	10
5.7	Full Python solution	11
6	What to carry forward	12
7	Complete upsolve notes	13
7.1	A. Same Parity	13
7.2	B. XOR Balanced Splits	13
7.3	C. Same Residue	13

1 Editorial overview

1.1 Problem-specific proof map

A. Same Parity. Adding or subtracting two preserves parity, and any two integers of equal parity differ by a multiple of two; parity is therefore both necessary and sufficient.

B. XOR Balanced Splits. A split is balanced when prefix xor equals suffix xor. Since `prefix xor suffix = total`, this occurs exactly when the prefix xor equals `total`; one left-to-right scan counts those positions in $O(n)$.

C. Same Residue. Every operation changes a value by a multiple of `k`, so residue modulo `k` is invariant. Equal residues are sufficient because each difference can be repaired by repeated `k`-steps.

D. Adjacent Flip Distance. The desired mismatch at the first unresolved position forces whether the adjacent operation starting there is used. A left-to-right XOR recurrence both constructs the only reduced operation set and verifies the final position.

1.2 Detailed solution guide

A. Parity Invariant. Each allowed operation changes the state but preserves the parity quantity named in the statement. Therefore states with different parity are unreachable from each other. To prove sufficiency, show a constructive sequence that reaches a canonical state for every configuration with the same parity. The answer is then determined entirely by that invariant.

B. Prefix XOR Splits. XOR is its own inverse, so the XOR of subarray `[l,r]` is `pref[r] xor pref[l-1]`. A split condition can therefore be checked by comparing prefix values rather than recomputing ranges. The algorithm scans the array while maintaining the prefix values that make the remaining segments possible.

C. Modulo Residue Invariant. If every operation changes a value by a multiple of `m`, then the residue modulo `m` is fixed. Conversely, if two values have the same residue, their difference is divisible by `m`, so the operation can bridge the gap in steps of size `m`. This gives both the necessary and the sufficient direction.

Common pitfalls. An invariant proof has two halves: impossibility for different invariant values and reachability or optimality within the same invariant class.

The solutions in this section are intentionally short because the hard work is the invariant.

- A uses parity: changing a value by 2 preserves parity, and equal parity is also sufficient.
- B uses prefix xor. A split has equal left and right xor exactly when the prefix xor matches the suffix xor.
- C uses residues modulo `k`: operations changing a number by multiples of `k` cannot alter its residue.

Each proof has both halves: the invariant is necessary because operations preserve it, and sufficient because a direct transformation or count follows once the invariant condition holds.

2 A. Same Parity

2.1 Restatement

Can all numbers become equal if one operation changes a number by 2?

2.2 Observations and algorithm

The proof map above states the invariant or reduction for this task. The implementation mirrors that argument: every branch is one of its explicit mathematical cases.

2.3 Correctness proof

The task-specific invariant in the proof map holds at initialization and is preserved by every stated transition. The final state therefore represents the requested answer. $\square$

2.4 Complexity

The bound is the concrete one stated for this task in the proof map; the code implements exactly its enumerations, transitions, or queries.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>
```

2.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    a=data[1:]; print("YES" if all(x%2==a[0]%2 for x in a) else "NO")
if __name__=="__main__": main()
```

3 B. XOR Balanced Splits

3.1 Restatement

Count split positions where left xor equals right xor.

3.2 Observations and algorithm

The proof map above states the invariant or reduction for this task. The implementation mirrors that argument: every branch is one of its explicit mathematical cases.

3.3 Correctness proof

The task-specific invariant in the proof map holds at initialization and is preserved by every stated transition. The final state therefore represents the requested answer. $\square$

3.4 Complexity

The bound is the concrete one stated for this task in the proof map; the code implements exactly its enumerations, transitions, or queries.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>
```

3.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    a=data[1:]; total=0
    for x in a: total^=x
    pref=ans=0
    for x in a[:-1]:
```

```
    pref^=x
    if pref==(total^pref): ans+=1
    print(ans)
if __name__=="__main__": main()
```

4 C. Same Residue

4.1 Restatement

Can all numbers become equal using operations that change a number by k ?

4.2 Observations and algorithm

The proof map above states the invariant or reduction for this task. The implementation mirrors that argument: every branch is one of its explicit mathematical cases.

4.3 Correctness proof

The task-specific invariant in the proof map holds at initialization and is preserved by every stated transition. The final state therefore represents the requested answer. $\square$

4.4 Complexity

The bound is the concrete one stated for this task in the proof map; the code implements exactly its enumerations, transitions, or queries.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;long long
```

4.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,k=data[0],data[1]; a=data[2:]; print("YES" if len({x%k for x in a})==1 else "NO")
if __name__=="__main__": main()
```


5 D. Adjacent Flip Distance

5.1 Restatement and first invariant

Two binary strings A and B have equal length n . One operation chooses an adjacent pair and flips both bits. Find the minimum number of operations needed to turn A into B , or -1 if it is impossible. Here n can be 200,000, so searching over strings is not viable.

Every operation changes exactly two bits, so the parity of the number of ones is invariant. Equivalently, the XOR of all bits is invariant. This gives a necessary condition, but it does not yet identify a shortest construction. The path shape of the allowed pairs gives a stronger left-to-right recurrence.

5.2 Forced recurrence and trace

Let $d[i] = A[i] \text{ xor } B[i]$; it says whether position i must be flipped an odd number of times. After cancelling pairs of identical operations, use each edge operation zero or one time. Let $x[i]$ denote whether the operation flipping positions i and $i+1$ is used.

Position 1 is touched only by $x[1]$, so $x[1] = d[1]$. For every interior position, the two touching operations must satisfy:

```
x[i-1] xor x[i] = d[i]
x[i] = d[i] xor x[i-1]
```

For $A=0000$ and $B=1001$, the mismatch vector is $1,0,0,1$:

```
x[1] = 1
x[2] = 0 xor 1 = 1
x[3] = 0 xor 1 = 1
```

The final position is touched only by $x[3]$; it needs $d[4]=1$, so this construction works and uses three operations. If the final check disagrees, no choice remains to repair it.

5.3 Algorithm

```
previous_operation = 0
answer = 0
for i from 1 to n-1:
    mismatch = A[i] xor B[i]
    current_operation = mismatch xor previous_operation
```

```

    answer += current_operation
    previous_operation = current_operation

if previous_operation != (A[n] xor B[n]):
    print -1
else:
    print answer

```

For $n=1$, the loop is empty and the final check correctly says that only identical strings are reachable.

5.4 Correctness proof

Cancelling two uses of the same adjacent operation changes no bit, so an optimal sequence uses each operation at most once. In such a sequence, position 1 forces $x[1]=d[1]$. Assuming $x[i-1]$ is forced, position i is correct exactly when $x[i-1] \text{ xor } x[i]=d[i]$, which forces the stated value of $x[i]$. By induction, the algorithm produces the only possible reduced choice for all operations through $x[n-1]$.

The final position is touched only by $x[n-1]$. If its value differs from $d[n]$, no reduced sequence can reach B, so -1 is correct. Otherwise every position satisfies its mismatch equation, so applying exactly the selected operations transforms A into B. Any sequence has the same forced parity for each operation; after cancellation it has exactly this selected set, so no sequence can use fewer operations. The reported count is therefore minimal. QED.

5.5 Complexity, implementation notes, and tests

The scan is $O(n)$ time and $O(1)$ extra space. Treat 0 and 1 characters as bits; no integer conversion of the entire strings is necessary. Test $n=1$, an odd mismatch count, endpoint-only mismatches, and a target produced by flipping alternating adjacent pairs. A common mistake is to check only the global XOR invariant and assume that it also constructs the minimum.

5.6 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    string start, target;
    if (!(cin >> n >> start >> target)) return 0;

    int previous_operation = 0;

```

```

int answer = 0;
for (int index = 0; index + 1 < n; index++) {
    int mismatch = start[index] != target[index];
    int current_operation = mismatch ^ previous_operation;
    answer += current_operation;
    previous_operation = current_operation;
}

if (previous_operation != (start[n - 1] != target[n - 1])) cout << -1 << '\n';
else cout << answer << '\n';
return 0;
}

```

5.7 Full Python solution

```

import sys

def main() -> None:
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    n = int(data[0])
    start = data[1]
    target = data[2]

    previous_operation = 0
    answer = 0
    for index in range(n - 1):
        mismatch = start[index] != target[index]
        current_operation = int(mismatch) ^ previous_operation
        answer += current_operation
        previous_operation = current_operation

    if previous_operation != (start[-1] != target[-1]):
        print(-1)
    else:
        print(answer)

if __name__ == "__main__":
    main()

```

6 What to carry forward

7 Complete upsolve notes

7.1 A. Same Parity

An operation changes a value by two, so parity is invariant. Mixed parity makes equality impossible. If all residues match, move every value by repeated ± 2 steps to a common value, proving sufficiency. Scan residues once: $O(n)$ time, $O(1)$ memory. Test one value, all even, all odd, and one mixed pair.

7.2 B. XOR Balanced Splits

Let total XOR be T and a prefix XOR be P . The suffix is $T \oplus P$, so equality is equivalent to $T=0$. Thus answer is $n-1$ when total XOR is zero, otherwise zero. This is $O(n)$; test $n=1$ and zero/nonzero totals. Do not recompute suffix XOR for every cut.

7.3 C. Same Residue

Adding or subtracting k preserves residue modulo k . Equal residues are also sufficient because values differ by multiples of k . Compare every $a[i] \% k$ to the first; normalize negatives in a generalized C++ version. $O(n)$ time, $O(1)$ memory; test $k=1$ and a mixed-residue pair.

Use `long long` for values in C++ and direct integers in Python. Run the local suite and brute-force very small states to test the claimed reachability class.

Red-level improvement comes from making the invariant explicit, stress-testing it, and then upsolving the stronger external versions.

For D, tiny tests use BFS over all 2^n binary strings to obtain the true shortest distance, not a copy of the recurrence. The scale case uses 200,000 bits and a directly known all-edge operation pattern.